Introduction to Topic Modelling on medical reports

Tawanda Gallan Chakuvinga

2026-03-21

Contents

1 Introduction

This work aims to provide an introductory concept of Topic modelling a branch of Natural Language Processing (NLP) in the medical field. Topic modelling also known as the unsupervised machine learning (UML) is a way of establishing trends, patterns in the text such that topics can be drawn from the textual data. We advance the mining of data in an under-researched field by means of exploring rich data (text) encountered in public health which may be readily accessible and analyzable by various stakeholders.

However, in this study, we will make use of AI generated medical health reports for very limited patients for the purposes of rolling out the introduction to topic modelling.

1.1 Study objectives

Firstly, AI generation and compilation of structured and unstructured data (text) from case (medical) reports from a given time frame that can be incorporated into a text or csv file to handle diverse datasets and easily accessible for data and text analysis by epidemiologists, public health researchers, and biostatisticians with expertise in the concerned fields, data science, quantitative, and computational skills personnel. These files consist of six columns, for each patient (a row); the year, signs and symptoms, diagnosis, treatment, prognosis and follow-ups made.

Secondly, to carry out Natural Language Processing (NLP) to perform topic modelling in order to observe the most prevalent signs and symptoms, diagnosis, treatment, prognosis and follow-ups in the health institution(s) of the given region.

2 Text preprocessing

We commence by loading our data compiled in the text file titled: *patient_data.txt* in R studio. A snapshot of the data is as seen in Figure 1.

```
data<- read.delim("patient_data.txt", header = TRUE, sep = "\t", stringsAsFactors = FALSE, encoding = "
```

```
View(data)
```

	Year	Signs.and.Symptoms	Diagnosis	Treatment	Progn
1	2018	Swollen-lymph-nodes	HIV	Counseling and support, Antiretroviral therapy (ART), Preven...	Chroni
2	2016	Muscle-cramps, Low-blood-pressure	cholera	Intravenous fluids	Usuall
3	2020	Fatigue, Fever, Chestpain, Cough that lasts more than three ...	tb	Antibiotics, Directly Observed Therapy (DOT), Hospitalization	Risk of
4	2016	Fatigue, Swollen-lymph-nodes, Diarrhea, Weightloss	HIV	Preventive medications, Antiretroviral therapy (ART)	Can le
5	2011	Coughing up blood or sputum, Chestpain, Fatigue, Fever, Ni...	tb	Hospitalization	Risk of
6	2013	Changes-in-bowel-bladder-habits, Persistent-cough, Chang...	cancer	Chemotherapy, Surgery	Progn
7	2015	Coughing up blood or sputum, Fever, Nightsweats, Loss-of-...	tb	Hospitalization, Antibiotics	Risk of
8	2018	Nightsweats, Fatigue, Loss-of-appetite, Cough that lasts mo...	tb	Hospitalization	Curabl
9	2015	Sweats, Fever	malaria	Antimalarial medications, Rest	Risk of
10	2014	Fatigue, Coughing up blood or sputum, Loss-of-appetite, C...	tb	Hospitalization, Antibiotics	Curabl

Figure 1: Snapshot

Note: The full text pre-processing code is covered in *data_preprocessing.R* and the data is in *patient_data.txt* and/ or *patient_data.csv*.

Below is the summary of the data we are working with

```
summary(data)
```

```
##       Year      Signs.and.Symptoms  Diagnosis          Treatment
##  Min.   :2010   Length:50          Length:50          Length:50
##  1st Qu.:2013   Class :character   Class :character   Class :character
##  Median :2016   Mode  :character   Mode  :character   Mode  :character
##  Mean   :2016
##  3rd Qu.:2019
##  Max.   :2020
```

```
##   Prognosis          Follow.ups
##  Length:50          Length:50
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
```

2.1 Packages

For simplicity, we are going to focus on the "signs and symptoms" column for each patient (row). We are going to perform topic modelling only on this column. Let's start by loading necessary packages:

```
# install packages first
library(tm)
library(topicmodels)
library(LDAvis)
library("stringr")
library(knitr)
library(kableExtra)
library(DT)
library(tm)
library(topicmodels)
library(reshape2)
library(ggplot2)
library(wordcloud)
library(pals)
library(SnowballC)
library(lda)
library(ldatuning)
library(flextable)
library(stringr)
library(quanteda)
library(textstem)
library(stringr)
library(dplyr)
```

To learn more, see Available packages by name and how to install R packages on the application and installation of the packages.

2.2 The corpus

In a text mining context, a corpus is plural of corpora which is a large, structured, and machine-readable collection of texts, audios or videos used for analysis. We move on to load our data into a corpus for the column Signs and Symptoms:

```
# Create a corpus from the documents
corpus <- Corpus(VectorSource(data$Signs.and.Symptoms))
```

Now that we have the corpus, we start by pre-processing the textual data in order to remove what may result in noise terms as we model. Text pre-processing involves and is not limited to the following:

-lowering all cases to lower cases as R is cursor sensitive which may analyse the same words differently since these may contain different cases,

```
corpus <- tm_map(corpus, content_transformer(tolower))
```

-removing all punctuation (full stops, commas, question marks, exclamation marks, colons, etc,), these are regarded as meaningless as they don't assist in anyway in topic modeling,

```
corpus <- tm_map(corpus, removePunctuation)
```

-removing all English stop words, these entail question words (where, how, when, who, etc.), articles (the, a, an),

```
corpus <- tm_map(corpus, removeWords, stopwords("en"))
```

-removing numbers,

```
corpus <- tm_map(corpus, removeNumbers)
```

-tokenization, a process of separating words from sentences and paragraphs into individual bag of words, among other form of pre-processing

```
corpus <- tm_map(corpus, stripWhitespace)
```

We then move on to construct a document term matrix (dtm) of the corpus. This step is necessary for converting unstructured text into a structured, numerical format that can easily be processed by computers and useful for mathematical and statistical analysis.

```
dtm <- DocumentTermMatrix(corpus)
```

The dtm has 50 rows (patients) ad 41 different terms (signs and symptoms) chopped to individual words.

```
dtm
```

```
## <<DocumentTermMatrix (documents: 50, terms: 41)>>
## Non-/sparse entries: 252/1798
## Sparsity           : 88%
## Maximal term length: 27
## Weighting          : term frequency (tf)
```

2.2.1 Preprocessed data

Consider the dtm comprising of pre-processed signs and symptoms for the first 5 patients in Figure 2.

```
inspect(dtm)
```

We get to explore the (dtm) matrix for the first five patients and the frequency of the first five terms.

```
[1] swollenlymphnodes
[2] musclecramps lowbloodpressure
[3] fatigue fever chestpain cough lasts three weeks nightsweats coughing blood sputum
[4] fatigue swollenlymphnodes diarrhea weightloss
[5] coughing blood sputum chestpain fatigue fever nightsweats cough lasts three weeks
```

Figure 2: First 5 patients

```
# Convert DTM to a matrix of word frequencies
# View the first 5 rows and 5 columns
as.matrix(dtm[1:5, 1:5])
```

```
##     Terms
## Docs swollenlymphnodes lowbloodpressure musclecramps blood chestpain
##    1                 1                0           0     0         0
##    2                 0                1           1     0         0
##    3                 0                0           0     1         1
##    4                 1                0           0     0         0
##    5                 0                0           0     1         1
```

According to the dtm, patient (Docs) 2, had "low blood pressure" and "muscle cramps" as the signs and symptoms (Terms). Some terms are lumped up together deliberately, otherwise their meaning and purpose would be lost. For instance, if "lowbloodpressure" were to be tokenized to "blood", "pressure", "low", we wouldn't know whether the "low" is for pressure or temperature, the same goes for other terms.

We move on to convert our dtm to a matrix of frequency of words so that we are able to understand the frequencies of the signs and symptoms as presented below for the 9 most frequent signs and symptoms;

```
# Convert DTM to a matrix of word frequencies
word_freq <- colSums(as.matrix(dtm))
# Sort the words by frequency
sorted_word_freq <- sort(word_freq, decreasing = TRUE)
# Plot the top 20 most frequent words
n<-50
top_words <- head(sorted_word_freq, n)
#word_freq
#sorted_word_freq
#top_words
head(sorted_word_freq,9)
```

```
##              fatigue                fever unexplainedweightloss
##                   27                   17                   11
##                cough              vomiting       persistentcough
##                    9                    9                    8
##                blood             coughing               sputum
##                    7                    7                    7
```

Fatigue is the most frequent sign and symptom recorded from the patients. This could easily be viewed from the bar plot of the frequency of terms as shown in Figure 3

```
# Plot a bar chart of word frequencies
barplot(top_words, las = 2, col = "lightblue", main = "Top Most Frequent Words", xlab = "Words", ylab =
```

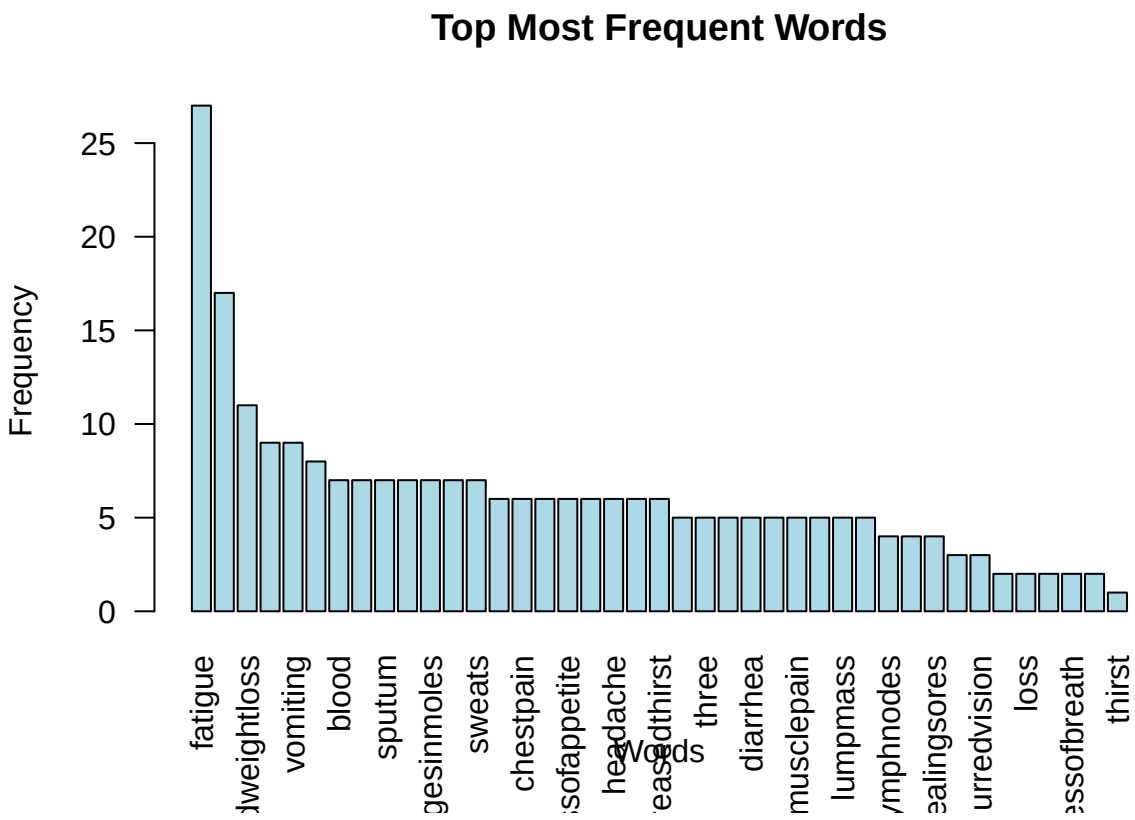

Figure 3: Bar plot

Alternatively, we can view it as a word cloud Figure 4:

```
wordcloud(names(top_words), freq = top_words, scale=c(3,0.5), min.freq = 1, max.words = 50, random.orde
```

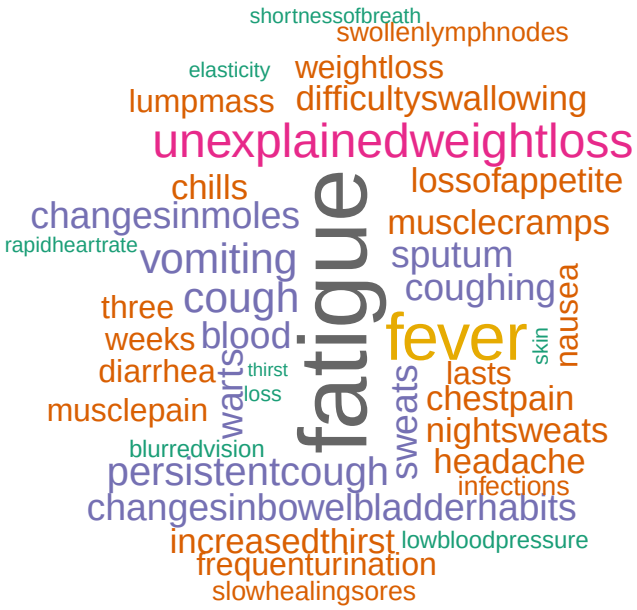

Figure 4: Word Cloud

These pictorials enable us to have a picture of the most common signs and symptoms in the medical institution or region under consideration. This would be instrumental in the distribution of resources and the quantities likely needed in the institution(s) with time. Moreover, the frequencies can be an early indicator of any brewing pandemics.

3 Model formulation

Now we move on to formulate our topic model, we will build the Latent Dirichlet Allocation (LDA) topic model, for more on the LDA (Blei, Ng, and Jordan 2003).

The formulation of topic models intends to ascertain the proportionate composition of number of topics in the medical reports for each patient. We may adjust parameters to establish suitable variables for our analysis.

Since LDA is a parameterized model, the number of topics, k, we consider is crucial beforehand. The optimal k has to be selected because a small k results in few abstract topics too complex to understand and too large a k results in many repetitive and redundant topics.

3.1 The model

We note that the dimensions of the dtm has the 50 patients and the 41 unique terms (signs and symptoms)

```
dim(dtm)
```

```
## [1] 50 41
```

It is worth noting that it may happen that after the processing and cleaning of our textual data, some rows or patients are left with no symptoms at all depending on how rigorous the cleaning process was. It is crucial to remove all rows with zero entries, implying no signs and symptoms associated with the patients prior to formulating the model.

```
sel_idx <- slam::row_sums(dtm) > 0
dtm <- dtm[sel_idx, ]
data <- data[sel_idx, ]
```

3.2 Optimal number of topics, *k*.

There are many approaches for determining the optimal number of topics to be considered before the model, in our work, we already know the 6 topics, which are the unique diagnoses from the data, these are; TB, HIV, Cancer, Diabetics and Cholera. However, for the sake of exploring our data, we will settle for 5 topics instead. We will only outline a two in one method for determining the optimal k, by using the (CaoJuan2009 and Deveaud2014) metrics Murzintcev (n.d.). In our situation we still stick to the CaoJuan2009 and Griffith2004 methods. The optimal k would show minimal values for CaoJuan2009 and at the same time, maximum values for Griffith2004.We plot the results as shown in Fig. 5, ideally, the optimal k would have been 2 topics only according to the data using this method followed by 6 topics which is trivial. It must be noted that other methods can still be applied.

```
# create models with different number of topics
result <- ldatuning::FindTopicsNumber(
  dtm,
  topics = seq(from = 2, to = 20, by = 1),
  metrics = c("CaoJuan2009",  "Deveaud2014"),
  method = "Gibbs",
  control = list(seed = 77),
  verbose = TRUE
)
```

```
## fit models... done.
## calculate metrics:
##   CaoJuan2009... done.
##   Deveaud2014... done.
```

```
FindTopicsNumber_plot(result)
```

4 The LDA model

For the sake of simplicity, we consider 5 topics *(k=5)* in formulating the LDA (topic model) model. We set a seed in order to obtain the same model each time we run the code. We set 500 iteration since we have minimal data for the purposes of illustrating the concept of topic modelling.

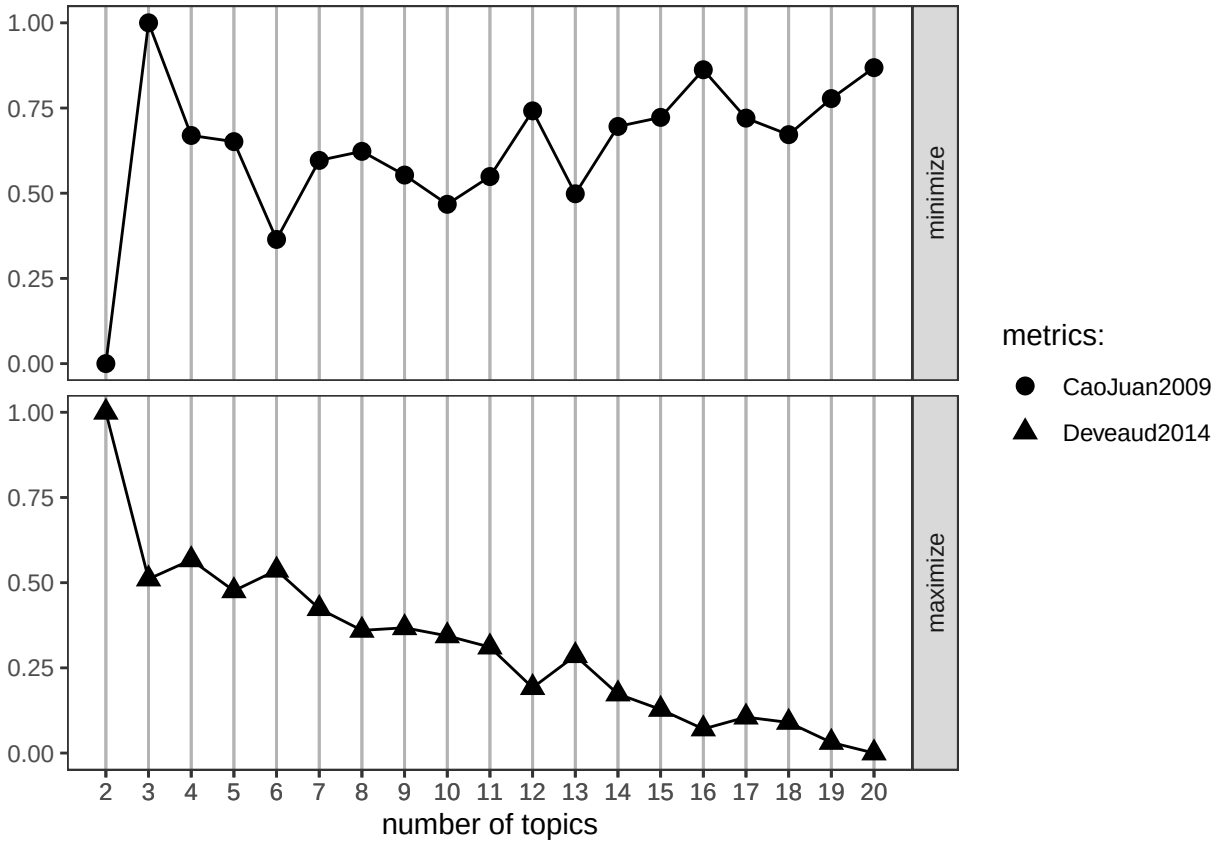

Figure 5: Optimal K value

```r
# number of topics
#K <- 5
K<-5
# set random number generator seed
set.seed(9161)
# compute the LDA model, inference via 1000 iterations of Gibbs sampling
topicModel <- LDA(dtm, K, method="Gibbs", control=list(iter = 500, verbose = 25))
```

```
## K = 5; V = 41; M = 50
## Sampling 500 iterations!
## Iteration 25 ...
## Iteration 50 ...
## Iteration 75 ...
## Iteration 100 ...
## Iteration 125 ...
## Iteration 150 ...
## Iteration 175 ...
## Iteration 200 ...
## Iteration 225 ...
## Iteration 250 ...
## Iteration 275 ...
## Iteration 300 ...
## Iteration 325 ...
## Iteration 350 ...
## Iteration 375 ...
## Iteration 400 ...
## Iteration 425 ...
## Iteration 450 ...
## Iteration 475 ...
## Iteration 500 ...
## Gibbs sampling completed!
```

The (topic) model inference results in two (approximate) posterior probability distributions: a distribution *theta* over k topics within each patient and a distribution *beta* over V terms (signs and symptoms) within each topic, where V represents the length of the vocabulary of the collection *(V = 41)*.

```r
# have a look a some of the results (posterior distributions)
tmResult <- posterior(topicModel)
# format of the resulting object
attributes(tmResult)
```

```
## $names
## [1] "terms"  "topics"
```

```r
ncol(dtm) # lengthOfVocab
```

```
## [1] 41
```

We get to see the dimensions of *beta*, probability distributions of the 5 topics over the entire vocabulary which add to one as expected. The dimensions of *beta* are 5 topics against each term. The beta probabilities for each topic on the associated terms have been saved in *beta_probabilities.csv*. Consider the probabilities for the 5 topics to five symptoms.

```
beta <- tmResult$terms   # get beta from results
dim(beta)  # K distributions over nTerms(DTM) terms
```

```
## [1]  5 41
```

```
#write.csv(beta, file = "beta_probabilities.csv", row.names = FALSE)
beta[1:5, 1:6]
```

```
##   swollenlymphnodes lowbloodpressure musclecramps       blood   chestpain
## 1       0.001996008      0.001996008  0.021956088 0.061876248 0.021956088
## 2       0.018003273      0.050736498  0.001636661 0.067103110 0.001636661
## 3       0.001848429      0.001848429  0.001848429 0.001848429 0.001848429
## 4       0.068736142      0.002217295  0.068736142 0.002217295 0.113082040
## 5       0.001610306      0.001610306  0.033816425 0.001610306 0.001610306
##         cough
## 1 0.001996008
## 2 0.001636661
## 3 0.094269871
## 4 0.024390244
## 5 0.049919485
```

For every patient we have a probability distribution of the topics, *theta*. The dimensions of *theta* are 50 patients against each topic. The *theta* probabilities for each patient on the associated topic have been saved in *theta_probabilities.csv*.

Consider the probabilities for the first 5 patients to the five topics, the dominant probability is on topic 4 for the first patient.

```
theta <- tmResult$topics
dim(theta)               # nDocs(DTM) distributions over K topics
```

```
## [1] 50  5
```

```
#rowSums(theta)[1:10]     # rows in theta sum to 1
#write.csv(theta, file = "theta_probabilities.csv", row.names = FALSE)
theta[1:3, 1:5]
```

```
##           1         2         3         4         5
## 1 0.1960784 0.1960784 0.1960784 0.2156863 0.1960784
## 2 0.1923077 0.2115385 0.1923077 0.1923077 0.2115385
## 3 0.1967213 0.2295082 0.2131148 0.1967213 0.1639344
```

The 5 most likely terms within the term probabilities *beta* of the inferred topics (with only the first 5) shown below.

```
terms(topicModel, 5)
```

```
##      Topic 1      Topic 2          Topic 3       Topic 4
## [1,] "sweats"     "fatigue"        "fever"       "fever"
## [2,] "fatigue"    "sputum"         "cough"       "changesinbowelbladderhabits"
```

```
## [3,] "three"      "lossofappetite" "nightsweats" "chestpain"
## [4,] "nausea"     "lumpmass"       "warts"       "fatigue"
## [5,] "musclepain" "blood"          "chills"      "weightloss"
##      Topic 5
## [1,] "unexplainedweightloss"
## [2,] "vomiting"
## [3,] "persistentcough"
## [4,] "changesinmoles"
## [5,] "slowhealingsores"
```

```
exampleTermData <- terms(topicModel, 10)
#exampleTermData[, 1:3]
```

Beyond looking into the topics, we visualize the topic distributions for each patient (in this case three randomly selected patients, patients 4, 10, 28). The plot, Fig. 6 shows that the first patient considered (patient 4) has signs and symptoms highly attributed to topic 4. The signs and symptoms of the patient 4, 10 and 28 are as follows:

```
exampleIds <- c(4, 10, 28)
#lapply(corpus[exampleIds], as.character)

#exampleIds <- c(4, 10, 28)
print(paste0(exampleIds[1], ": ", substr(content(corpus[[exampleIds[1]]]), 0, 400), '...'))
```

```
## [1] "4: fatigue swollenlymphnodes diarrhea weightloss..."
```

```
print(paste0(exampleIds[2], ": ", substr(content(corpus[[exampleIds[2]]]), 0, 400), '...'))
```

```
## [1] "10: fatigue coughing blood sputum lossofappetite chestpain fever nightsweats cough lasts three
```

```
print(paste0(exampleIds[3], ": ", substr(content(corpus[[exampleIds[3]]]), 0, 400), '...'))
```

```
## [1] "28: fatigue..."
```

The probabilities of each patient having the symptoms associated with each topic are shown, patient 4 is most likely to be categorized (or have ailment) in topic 4.

```
##### Alterations
# Get the number of topics
num_topics <- ncol(theta)
num_topics
```

```
## [1] 5
```

```
# Define topicNames based on the number of topics
topicNames <- paste("Topic", 1:num_topics)
topicNames
```

```
## [1] "Topic 1" "Topic 2" "Topic 3" "Topic 4" "Topic 5"
```

```
# Now proceed with the rest of your code
# Get topic proportions from example documents
topicProportionExamples <- theta[exampleIds,]
topicProportionExamples
```

```
##            1         2         3         4         5
## 4  0.1851852 0.1851852 0.2037037 0.2222222 0.2037037
## 10 0.1935484 0.2580645 0.1774194 0.1935484 0.1774194
## 28 0.1960784 0.2156863 0.1960784 0.1960784 0.1960784
```

```
####
```

```
N <- length(exampleIds)
# get topic proportions form example documents
topicProportionExamples <- theta[exampleIds,]
colnames(topicProportionExamples) <- topicNames
vizDataFrame <- melt(cbind(data.frame(topicProportionExamples), document = factor(1:N)), variable.name =
ggplot(data = vizDataFrame, aes(topic, value, fill = document), ylab = "proportion") +
  geom_bar(stat="identity") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  coord_flip() +
  facet_wrap(~ document, ncol = N)
```

```
## Warning in fortify(data, ...): Arguments in `...` must be used.
## x Problematic argument:
## * ylab = "proportion"
## i Did you misspell an argument name?
```

In Fig. 6, topic 4 has keywords: *muscle cramps, diarrhea, weightloss, swollen lymphnodes, low blood pressure* highly linked to cholera, it is highly likely that the first patient (patient 4) had cholera.

Taking a look at the LDA model once again, considering the value of alpha=0.2 since we are dealing with limited number of topics, in order to avoid overriding among the topics.

```
# see alpha from previous model
attr(topicModel, "alpha")
```

```
## [1] 10
```

```
#The new alpha=0.2
# set random number generator seed
set.seed(91)
topicModel2 <- LDA(dtm, K, method="Gibbs", control=list(iter = 500, verbose = 25, alpha = 0.2))
```

```
## K = 5; V = 41; M = 50
## Sampling 500 iterations!
## Iteration 25 ...
## Iteration 50 ...
## Iteration 75 ...
## Iteration 100 ...
## Iteration 125 ...
```

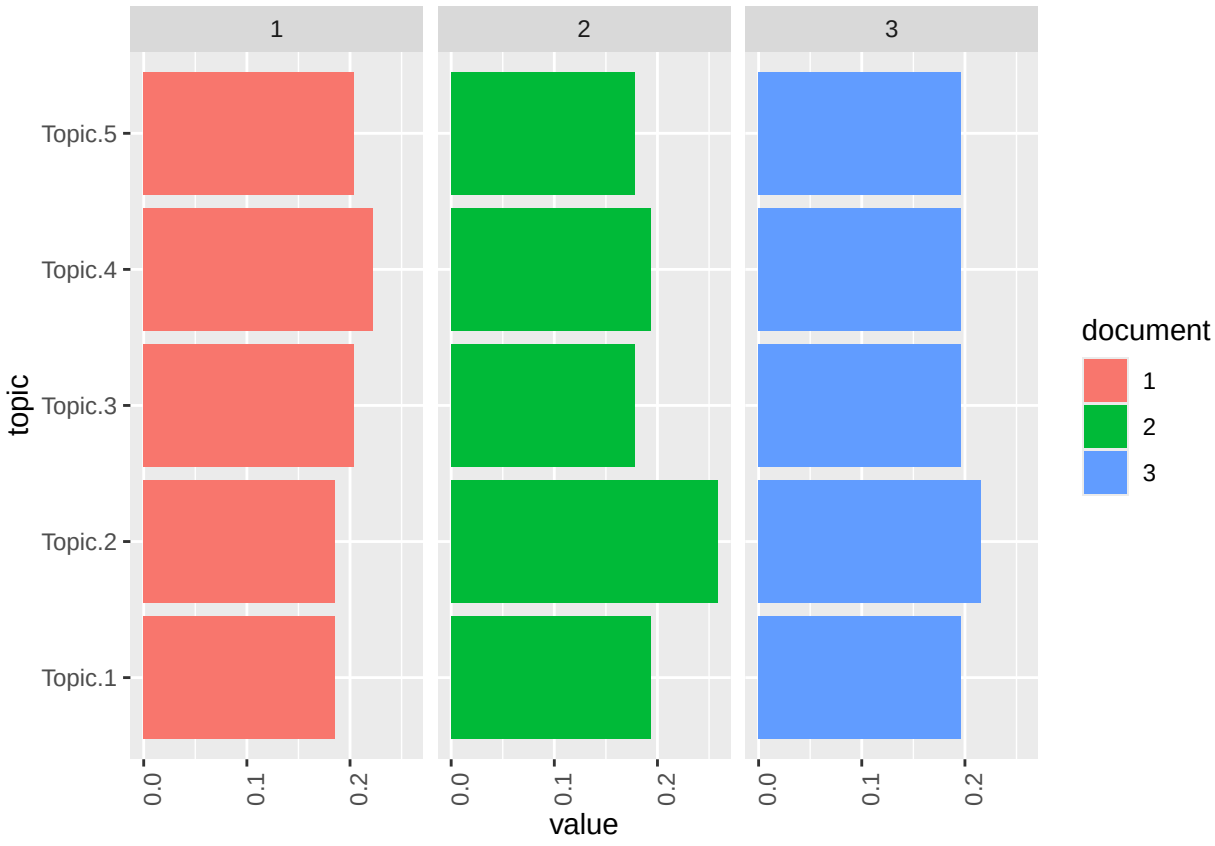

Figure 6: Three patients' topic probabilities

```
## Iteration 150 ...
## Iteration 175 ...
## Iteration 200 ...
## Iteration 225 ...
## Iteration 250 ...
## Iteration 275 ...
## Iteration 300 ...
## Iteration 325 ...
## Iteration 350 ...
## Iteration 375 ...
## Iteration 400 ...
## Iteration 425 ...
## Iteration 450 ...
## Iteration 475 ...
## Iteration 500 ...
## Gibbs sampling completed!
```

```
tmResult <- posterior(topicModel2)
theta <- tmResult$topics
beta <- tmResult$terms
topicNames <- apply(terms(topicModel2, 5), 2, paste, collapse = " ")  # reset topicnames
```

4.1 Topic distribution

The topic distribution within a patient can be controlled with the Alpha-parameter of the model. Higher alpha priors for topics result in an even distribution of topics within a patient. Low alpha priors ensure that the inference process distributes the probability mass on a few topics for each patient. We set the alpha prior to a lower value of 0.2 to see how this affects the topic distributions in the model. We see the difference in the distribution structure of the topic as visualized below, Fig. 7:

```
#Now visualize the topic distributions in the three patients again.
#What are the differences in the distribution structure?
topicProportionExamples <- theta[exampleIds,]
colnames(topicProportionExamples) <- topicNames
```

```
vizDataFrame <- melt(cbind(data.frame(topicProportionExamples), document = factor(1:N)), variable.name =
ggplot(data = vizDataFrame, aes(topic, value, fill = document)) + #, ylab = "proportion") +
  geom_bar(stat="identity") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1),
    legend.position = "bottom") +
  coord_flip() +
  facet_wrap(~ document, ncol = N)
```

We express the topics comprising of what each entails;

```
# re-rank top topic terms for topic names
topicNames <- apply(lda::top.topic.words(beta, 5, by.score = T), 2, paste, collapse = " ")
topicNames
```

```
##                                                                                1
##                                                  "blood coughing sputum cough fever"
```

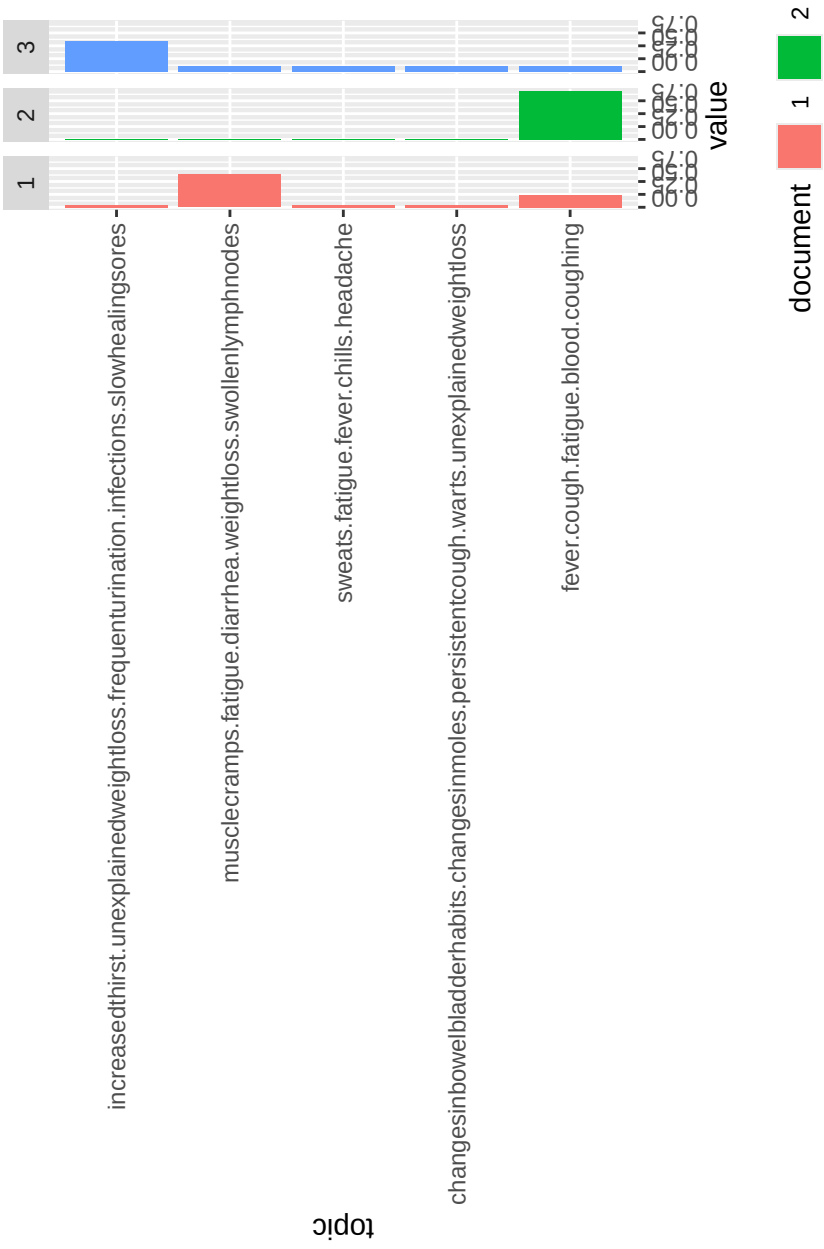

Figure 7: Selected patients alpha=0.2

```
##                                                                                 2
## "changesinbowelbladderhabits changesinmoles warts persistentcough difficultyswallowing"
##                                                                                 3
##                                                  "sweats chills headache musclepain nausea"
##                                                                                 4
##                              "musclecramps diarrhea weightloss swollenlymphnodes vomiting"
##                                                                                 5
##   "increasedthirst frequenturination unexplainedweightloss infections slowhealingsores"
```

4.2 Topic ranking

We aspire to obtain a more meaningful order of top signs and symptoms per topic by re-ranking them with a specific score (Chang et al. 2009). The idea of re-ranking terms is identical to the notion of TF-IDF. The least meaningful signs and symptoms in describing a topic are the ones which appear most with high probabilities. This implies that the scoring favours terms to describe a topic.

It is fundamental to identify the dominant topics within the case series (collection of patients' reports). We will consider two approaches in shaping the topics in a certain way.

4.2.1 Approach 1

Sorting topics with respect to their probabilities within the entire list of patients as shown in the output:

```
#nrow(dtm)
topicProportions <- colSums(theta) / nrow(dtm)  # mean probabilities over all
names(topicProportions) <- topicNames     # assign the topic names
sort(topicProportions, decreasing = TRUE) # show summed proportions in decreased order
```

```
##                                                    blood coughing sputum cough fever
##                                                                            0.2293311
##                              musclecramps diarrhea weightloss swollenlymphnodes vomiting
##                                                                            0.2095839
## changesinbowelbladderhabits changesinmoles warts persistentcough difficultyswallowing
##                                                                            0.2044648
##                                                 sweats chills headache musclepain nausea
##                                                                            0.2026712
##   increasedthirst frequenturination unexplainedweightloss infections slowhealingsores
##                                                                            0.1539490
```

or

```
#to 5 decimal places
soP <- sort(topicProportions, decreasing = TRUE)
paste(round(soP, 5), ":", names(soP))
```

```
## [1] "0.22933 : blood coughing sputum cough fever"
## [2] "0.20958 : musclecramps diarrhea weightloss swollenlymphnodes vomiting"
## [3] "0.20446 : changesinbowelbladderhabits changesinmoles warts persistentcough difficultyswallowing
## [4] "0.20267 : sweats chills headache musclepain nausea"
## [5] "0.15395 : increasedthirst frequenturination unexplainedweightloss infections slowhealingsores"
```

We observe that some topics are slightly likely to occur in the corpus than others. This rather describes general thematic coherence. Other topics may correspond more to specific contents.

4.2.2 Approach 2

Counting how frequently a topic appears as a primary topic within a patient. The method is also called Rank-1 as shown in the output.

```
countsOfPrimaryTopics <- rep(0, K)
names(countsOfPrimaryTopics) <- topicNames
for (i in 1:nrow(dtm)) {
  topicsPerDoc <- theta[i, ] # select topic distribution for document i
  # get first element position from ordered list
  primaryTopic <- order(topicsPerDoc, decreasing = TRUE)[1]
  countsOfPrimaryTopics[primaryTopic] <- countsOfPrimaryTopics[primaryTopic] + 1
}
sort(countsOfPrimaryTopics, decreasing = TRUE)
```

```
##                                                    blood coughing sputum cough fever
##                                                                                   12
## changesinbowelbladderhabits changesinmoles warts persistentcough difficultyswallowing
##                                                                                   11
##                           musclecramps diarrhea weightloss swollenlymphnodes vomiting
##                                                                                   11
##                                              sweats chills headache musclepain nausea
##                                                                                    9
##   increasedthirst frequenturination unexplainedweightloss infections slowhealingsores
##                                                                                    7
```

or inline

```
#inline
so <- sort(countsOfPrimaryTopics, decreasing = TRUE)
paste(so, ":", names(so))
```

```
## [1] "12 : blood coughing sputum cough fever"
## [2] "11 : changesinbowelbladderhabits changesinmoles warts persistentcough difficultyswallowing"
## [3] "11 : musclecramps diarrhea weightloss swollenlymphnodes vomiting"
## [4] "9 : sweats chills headache musclepain nausea"
## [5] "7 : increasedthirst frequenturination unexplainedweightloss infections slowhealingsores"
```

Evidently, sorting topics by the Rank-1 method places topics with rather specific thematic coherences in upper ranks of the list.

Topics sorting can be considered for advanced analysis for example, in the semantic interpretation of topics in a population of patients, time series analysis of the most important topics or the filtering of the original patient population based on specific sub-topics.

4.3 Filtering topics

Due to the fact that a topic model generates topic probabilities for each patient, it translates to possible use for thematic filtering of a patient population. As a filter, we select only those patients who exceed a certain threshold of their probability value for certain topics (for example, each patient who contains topic X which is more than 20 percent).

```
##Filtering
topicToFilter <- 5  # you can set this manually ...
# ... or have it selected by a term in the topic name (e.g. 'illness')
topicToFilter <- grep('sputum', topicNames)[1]
topicThreshold <- 0.2
selectedDocumentIndexes <- which(theta[, topicToFilter] >= topicThreshold)
filteredCorpus <- corpus[selectedDocumentIndexes]
# show length of filtered corpus
filteredCorpus
```

```
## <<SimpleCorpus>>
## Metadata:  corpus specific: 1, document level (indexed): 0
## Content:  documents: 13
```

Our filtered corpus contains 0 patients related to the specified topic.

4.4 Topic proportions over time

Finally, we provide a view on the topics in the patients' records over time. We aggregate mean topic proportions per year of all patients' sign and symptoms from the year 2010-2020. These aggregated topic proportions can then be visualized, e.g. as a bar plot, Fig.8

```
#data$Year <- paste0(substr(data$Year, 1, 4))
#View(textdata)
colnames(data)
```

```
## [1] "Year"               "Signs.and.Symptoms" "Diagnosis"
## [4] "Treatment"          "Prognosis"          "Follow.ups"
```

```
# get mean topic proportions per decade
topic_proportion_per_year <- aggregate(theta, by = list(year = data$Year), mean)
# set topic names to aggregated columns
colnames(topic_proportion_per_year)[2:(K+1)] <- topicNames
# reshape data frame
vizDataFrame <- melt(topic_proportion_per_year, id.vars = "year")
# plot topic proportions per decade as bar plot
ggplot(vizDataFrame, aes(x=year, y=value, fill=variable)) +
  geom_bar(stat = "identity") + ylab("proportion") +
  scale_fill_manual(values = paste0(alphabet(20), "FF"), name = "topic")+
  theme(axis.text.x = element_text(angle = 90, hjust = 1),
    legend.position = "bottom")
```

It appears the topic on "sweats, chills, increased thirst, muscle pain and nausea" which could be signs and symptoms of diabetics were on a surge at the beginning and the end of the time frame considered.

5 Discussion and Conclusion

Natural language processing NLP assist in generating visual representations and provide insight into the prevalent signs and symptoms within a medical institution or a specific region. They play a crucial role in

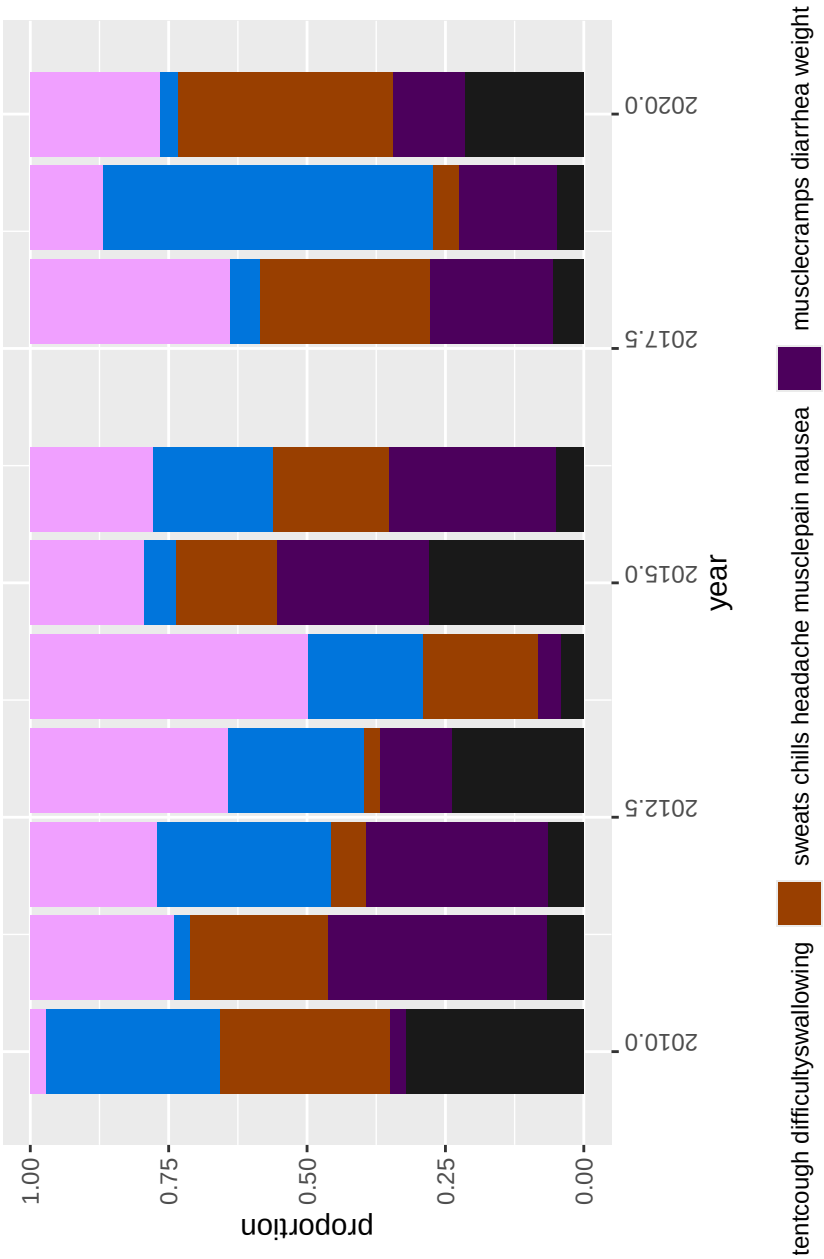

Figure 8: Time series bar-plot

resource allocation by helping estimate the required quantities of medical supplies and personnel. Additionally, the observed frequencies serve as potential indicators of emerging pandemics.

On the other hand, topic modelling is instrumental in determining not only the most prevalent signs and symptoms, but can be expanded to investigate the types of treatment offered to patients, the prognosis associated with the quality of follow up on the patients. In my view, this provides an extremely rare method that can be implemented by biostatisticians in data triangulation and evidence synthesis. Specifically, incorporating qualitative synthesis which involves synthesizing qualitative evidence from open-ended questions, interviews, case reports (which we partially covered in this presentation) to compliment quantitative findings. Qualitative synthesis includes thematic analysis, narrative synthesis and meta-enthography. There are more text analysis techniques beneficial to the medical field which include sentiment analysis that can be conducted on patients' feedback and follow up, and prognosis. Additionally, text classification using supervised machine learning can be conducted to classify patients' prognosis based on the treatment and follow-ups.

Various types of models can be developed that can be used for future classification of the state of patients' prognosis where patients were followed up on based on other previously related cases. The reports can be separated into two sets, the major for training a model and the minor for testing the model. We can develop a prognosis prediction model that is trained from the compiled medical reports meant for the model training (the training set) and these are associated with ailments of concern. The model would then be tested as a way of assessing its performance and we go on to conduct statistical analysis of its accuracy using the test set of the medical reports or other related documentation. Adjustments to improve its performance may be considered if necessary until the model appears satisfactory in making more accurate and reliable predictions on the prognosis of the given types of immune systems bearing the considered ailments. The formulated, tested and validated prognosis model would be easily employed to assist medical practitioners in giving them a second opinion on their own view of the prognosis of the ailments for specified immune systems, and in turn may help in decision making for the benefit of their patients. In short, the model would take in as its input, the history or medical report of patients and be able to predict the prognosis based on the description of their immune system and the ailment(s) from historical experience of other related patients to address public health challenges.

5.1 Limitation

-limited data for the purposes of rolling an introductory approach to NLP topic modelling,

-data not readily available in some set ups due limited (digitalisation) resources and sensitivity of data

-computationally expensive for massive data, especially at national level.

6 References

Blei, David M., Andrew Y. Ng, and Michael I. Jordan. 2003. "Latent Dirichlet Allocation." Journal of Machine Learning Research 3 (3): 993–1022.

Murzintcev, Nikita. n.d. "Select Number of Topics for LDA Model." https://cran.r-project.org/web/packages/ldatuning/vignettes/topics.html.

Chang, Jonathan, Sean Gerrish, Chong Wang, Jordan L. Boyd-graber, and David M. Blei. 2009. "Reading Tea Leaves: How Humans Interpret Topic Models." In Advances in Neural Information Processing Systems 22, edited by Yoshua Bengio, Dale Schuurmans, John D. Lafferty, Christopher K. Williams, and Aron Culotta, 288–96. Curran. http://papers.nips.cc/paper/3700-reading-tea-leaves-how-humans-interpret-topic-models.pdf.